Module 6 Assignment: Views and Materialized Views
CPT 166 – Fundamentals of SQL
Sylvia Schlotterbeck 11/2/25

1.Create a View: Customer Orders:

```
CREATE VIEW vw_customer_orders AS
  SELECT customer_name, order_id, order_date
  FROM orders
  LEFT JOIN customers ON customers.customer_id = orders.customer_id
;

SELECT *
FROM vw_customer_orders
ORDER BY order_date DESC
LIMIT 10;
```

	customer_name character varying (255)	order_id integer	order_date date
1	Simons bistro	11074	2023-05-06
2	Richter Supermarkt	11075	2023-05-06
3	Rattlesnake Canyon Grocery	11077	2023-05-06
4	Bon app	11076	2023-05-06
5	Ernst Handel	11072	2023-05-05
6	Lehmanns Marktstand	11070	2023-05-05
7	Pericles Comidas clasicas	11073	2023-05-05
8	LILA-Supermercado	11071	2023-05-05
9	Drachenblut Delikatessend	11067	2023-05-04
10	Queen Cozinha	11068	2023-05-04

Total rows: 10 | Query complete 00:00:00.079

2.Create a View: Product Sales Summary:

```
CREATE VIEW vw_product_sales_summary AS
  SELECT
    products.product_id,
     product_name,
     SUM(quantity) AS total_quantity,
     SUM(quantity) * price AS total_sales
  FROM
    products
  INNER JOIN order_details ON products.product_id =
order_details.product_id
  GROUP BY products.product_id;

SELECT *
FROM vw_product_sales_summary
ORDER BY total_sales DESC
LIMIT 5;
```

	product_id integer	product_name character varying (255)	total_quantity bigint	total_sales numeric
1	38	Cote de Blaye	623	164160.50
2	29	Thoringer Rostbratwurst	746	92347.34
3	59	Raclette Courdavault	1496	82280.00
4	60	Camembert Pierrot	1577	53618.00
5	62	Tarte au sucre	1083	53391.90

3.Create a View: Active Customers:

```
CREATE VIEW vw_active_customers AS
  SELECT
    customer_name, COUNT(order_id) AS number_of_orders
  FROM
    orders
  LEFT JOIN customers ON orders.customer_id = customers.customer_id
  GROUP BY customer_name
  HAVING COUNT(order_id) >= 5
  ORDER BY number_of_orders DESC
  ;

SELECT customer_name
FROM vw_active_customers
ORDER BY customer_name;
```

	customer_name character varying (255)
1	Alfreds Futterkiste
2	Antonio Moreno Taquera
3	Around the Horn
4	Berglunds snabbkoep
5	Blauer See Delikatessen
6	Blondel pere et fils
7	Bon app
8	Bottom-Dollar Marketse
9	Bs Beverages
10	Cactus Comidas para llevar
11	Chop-suey Chinese
12	Comercio Mineiro
13	Die Wandernde Kuh
14	Drachenblut Delikatessend
15	Eastern Connection
16	Ernst Handel
17	Familia Arquibaldo
18	Folies gourmandes
19	Folk och fe HB
20	Franchi S.p.A.
21	Frankenversand
22	Furia Bacalhau e Frutos do Mar
23	Galeria del gastronomo
24	Godos Cocina Tipica
25	Gourmet Lanchonetes

Total rows: 72 Query complete

4.Create a Materialized View: Monthly Sales:

```
CREATE MATERIALIZED VIEW mv_monthly_sales AS
  SELECT
    SUM(quantity * price) AS total_sales,
    EXTRACT(MONTH FROM order_date) AS month,
    EXTRACT(YEAR FROM order_date) AS year
  FROM orders
  LEFT JOIN order_details ON orders.order_id = order_details.order_id
  LEFT JOIN products ON order_details.product_id =
products.product_id
  GROUP BY year, month
  ORDER BY year, month
;

SELECT *
FROM mv_monthly_sales
ORDER BY year DESC, month DESC
LIMIT 6;
```

	total_sales numeric	month numeric	year numeric
1	19898.66	5	2023
2	134630.56	4	2023
3	109825.45	3	2023
4	104561.95	2	2023
5	100854.72	1	2023
6	77476.26	12	2022

5.Refresh Materialized View

* Run the appropriate command to refresh the materialized view.
* Add a short written explanation (2–3 sentences) of why refreshing is necessary:

```
REFRESH MATERIALIZED VIEW mv_monthly_sales;
```

My explanation: refreshing a materialized view (by running the ‘Refresh Materialized View’ command) is necessary when you know that the underlying data in the tables has updated, or when other context has altered and you want the most current data from the underlying tables. This is because unlike a VIEW, which isn’t stored on disk, and is run afresh every time it is called; a Materialized View is a query that runs once and that iteration of the query is stored to disk. So when you query a materialized view, you are accessing that stored query on the disk, not the tables that were queried when the Materialized View was created, so if you want to query the tables themselves, you must refresh the materialized view.

6. Short Write-Up on the difference between a view and a materialized view, and why you might choose one over the other:

I explained about the differences a bit in the previous paragraph, but in short, both Views and Materialized Views are stored queries that you can utilize when writing SQL statements to your database once they are created, in similar ways to the tables that make up the database. This can be helpful when there is a query you frequently use, to prevent you from having to type up the whole things each time, and is particularly helpful when that query gets longer and/or more complex.

The primary, important difference between Views and Materialized Views is that a View is stored as a simple SQL statement, and isn’t run when it is created, while a Materialized View is run when it is created, and won’t re-query the underlying tables when it is called in subsequent statments/queries unless it is refreshed. A Materialized View can be helpful when getting a snapshot of the tables queried at a particular time, when you want to be able to reference the database snapshot and have it remain stable until you want to refresh and take a snapshot at a later time. A View is helpful in a similar way to writing a function in another computer language like Python might be, in that you can utilize the view multiple times without having to re-write it, and it can dramatically cut-down the length of subsequent queries that make use of it.